



Proyecto Unidad 1 - Internet de las Cosas

SmartHome IoT - Sistema de automatización y monitoreo para una casa inteligente

Ricardo Pérez
riperez@utalca.cl

1. Introducción

La automatización del hogar es una de las aplicaciones más extendidas del Internet de las Cosas. Sistemas que controlan la iluminación, detectan presencia, monitorean la calidad del aire o alertan ante eventos anómalos ya forman parte de productos comerciales como Google Home, Amazon Alexa o Apple HomeKit.

En este proyecto, cada equipo deberá diseñar e implementar un **prototipo de casa inteligente** utilizando hardware real, comunicación MQTT, una plataforma de monitoreo en Node-RED y una estrategia de control automático. El sistema deberá ser capaz de reaccionar ante eventos del entorno de forma autónoma y mostrar toda la información en un *dashboard* accesible desde la red local.

2. Problema a resolver

Una familia desea automatizar su hogar para mejorar la seguridad, el confort y el ahorro energético. Para ello, solicita un sistema IoT que permita:

- Monitorear las condiciones ambientales del hogar (temperatura, humedad, calidad del aire).
- Detectar presencia o movimiento en áreas clave.
- Capturar imágenes para identificar si hay una persona presente y quién es esa persona.
- Recibir alertas automáticas ante condiciones anómalas.
- Controlar actuadores de forma automática y manual.
- Visualizar toda la información desde un *dashboard* centralizado.

3. Objetivo

Diseñar e implementar un prototipo IoT funcional que simule la automatización de una vivienda, integrando sensores físicos, comunicación mediante MQTT, visión artificial básica en Node-RED y control automático basado en reglas.

4. Objetivos específicos

1. Conectar y programar los sensores del kit asignado en una placa Arduino MKR y/o ESP32.
2. Utilizar la ESP32-CAM para capturar imágenes y detectar presencia de personas mediante Node-RED. Se recomienda usar TensorFlow.js.
3. Publicar los datos medidos al broker MQTT utilizando una jerarquía de tópicos ordenada.
4. Diseñar un *dashboard* en Node-RED que visualice todas las variables del sistema en tiempo real.
5. Implementar al menos dos estrategias de control automático basadas en umbrales o detección de eventos.
6. Integrar al menos una notificación externa (Telegram o correo electrónico) ante eventos críticos.
7. Registrar históricamente los datos en un archivo CSV o base de datos desde Node-RED.
8. Documentar la arquitectura, decisiones técnicas y resultados en un informe técnico.

5. Funcionalidades mínimas requeridas

5.1 Sensores y publicación MQTT

El nodo de sensores debe leer al menos **4 variables físicas** (incluyendo el sensor diferencial) y publicarlas al broker MQTT usando la siguiente jerarquía de tópicos:

```
smarthome/equipoXX/temperatura
smarthome/equipoXX/humedad
smarthome/equipoXX/gas
smarthome/equipoXX/movimiento
smarthome/equipoXX/sensor_extra
smarthome/equipoXX/alerta
smarthome/equipoXX/control/led
smarthome/equipoXX/control/buzzer
smarthome/equipoXX/camara/evento
```

Los mensajes deben enviarse en formato JSON. Ejemplo:

```
{
  "equipo": "equipoXX",
  "temperatura": 24.3,
  "humedad": 58,
  "gas": 310,
  "movimiento": true,
  "sensor_extra": 720,
  "alerta": "movimiento_detectado"
}
```

5.2 ESP32-CAM y detección de personas

- La ESP32-CAM debe transmitir imágenes (snapshot o stream MJPEG).
- Node-RED debe procesar las imágenes usando el nodo `node-red-contrib-tfjs-coco-ssd` para detectar si hay una persona en el encuadre.
- Si se detecta una persona, el sistema debe publicar un mensaje MQTT y activar una alerta en el *dashboard*.
- La imagen capturada debe mostrarse en el *dashboard*.

Nota técnica: La detección de personas no ocurre en la ESP32-CAM, sino en Node-RED usando TensorFlow.js. La cámara solo captura y transmite la imagen.

5.3 Dashboard en Node-RED

El *dashboard* debe mostrar como mínimo:

- Valores actuales de temperatura, humedad, gas y sensor diferencial.
- Estado de movimiento y detección de persona (sí/no).
- Estado de alerta del sistema.
- Imagen capturada por la ESP32-CAM.
- Botón para activar/desactivar actuadores manualmente.
- Botón para solicitar una captura manual de imagen.
- Indicador visual del estado general del sistema.
- Gráfico histórico de al menos una variable (temperatura o gas).

5.4 Control automático

El sistema debe implementar al menos **dos reglas de control automático**. Ejemplos:

Si temperatura > 30°C → encender LED de alerta + enviar notificación

Si gas > umbral definido → activar buzzer + capturar imagen
+ enviar notificación

Si movimiento Y persona detectada → registrar evento + alerta en dashboard

Si LDR < umbral (noche) Y movimiento → activar alerta

Las reglas deben estar implementadas en Node-RED y documentadas en el informe.

5.5 Notificaciones externas

El sistema debe enviar al menos **una notificación automática** ante un evento crítico, usando uno de los siguientes canales:

- **Telegram:** nodo `node-red-contrib-telegrambot`.
- **Correo electrónico:** nodo `node-red-node-email`.

La notificación debe incluir: tipo de evento, valor que lo disparó y *timestamp*.

5.6 Registro histórico

Los datos deben registrarse de forma continua en Node-RED usando:

- Nodo `node-red-node-csv` para exportar a archivo `.csv`, o
- Base de datos local con InfluxDB o SQLite.

El registro debe incluir al menos: *timestamp*, temperatura, humedad, nivel de gas y estado de alerta.

6. Entregables

a) Prototipo funcional

Sistema completo en funcionamiento, demostrable en sala durante la defensa.

b) Código fuente

- Código del nodo de sensores (Arduino MKR / ESP32).
- Código de la ESP32-CAM.
- Flujo de Node-RED exportado en formato `.json`.
- Todo debe estar disponible en un repositorio o carpeta ordenada.

c) Informe técnico (10–15 páginas)

El informe debe contener:

1. Portada (integrantes, equipo asignado, fecha).
2. Introducción y descripción del problema.
3. Objetivo general y objetivos específicos.
4. Diagrama de arquitectura del sistema.
5. Descripción de cada sensor utilizado y su rol en el sistema.
6. Estructura de tópicos MQTT con ejemplos de mensajes JSON.
7. Descripción del *dashboard* y capturas de pantalla.
8. Estrategias de control implementadas.
9. Descripción de notificaciones y registro histórico.
10. Problemas encontrados y soluciones aplicadas.
11. Conclusiones.

d) Defensa oral (15 minutos)

- **8 minutos:** demostración del sistema funcionando en vivo.
- **7 minutos:** preguntas técnicas del profesor.
- Todos los integrantes deben participar activamente.

12. Rúbrica de evaluación

La calificación final del proyecto se obtiene en escala de **1,0 a 7,0**. La nota mínima de aprobación es **4,0**. Cada criterio se evalúa en una escala interna de 1 a 4, ponderada según su peso relativo.

Criterio	Pond.	Logrado (4)	Parcial (2-3)	No logrado (1)
Sensores e integración	20 %	4 variables correctamente leídas y publicadas, incluyendo sensor diferencial	2-3 variables operativas o sensor diferencial sin integrar	Menos de 2 sensores funcionando
Comunicación MQTT	15 %	Broker activo, tópicos ordenados y mensajes en JSON	Comunicación parcial o tópicos sin jerarquía	No hay comunicación MQTT
ESP32-CAM y detección de personas	15 %	Stream activo y detección de personas en Node-RED funcionando	Cámara operativa pero sin detección, o detección con errores frecuentes	Cámara no funcional
Dashboard Node-RED	15 %	Dashboard completo con todas las variables, gráfico e imagen de cámara	Dashboard parcial, faltan variables o imagen	Sin dashboard
Control automático	10 %	2 reglas implementadas y verificables en demo	1 regla funcional	Sin control automático
Notificaciones y registro	10 %	Notificación externa activa y registro histórico guardando datos	Solo uno de los dos funciona	Ninguno implementado
Informe técnico	10 %	Completo, ordenado, con todos los elementos requeridos y capturas	Faltan secciones relevantes o capturas	Informe ausente o muy incompleto
Defensa oral	5 %	Todos participan, explican con claridad las decisiones técnicas	Participación desigual o explicaciones superficiales	Sin defensa o sin dominio técnico
Total	100 %			

Cuadro 1: Rúbrica de evaluación del proyecto

Conversión de puntaje a nota

El puntaje ponderado total se calcula sobre 100 puntos y se convierte a nota según la siguiente escala lineal, con nota mínima 1,0 y máxima 7,0:

$$\text{Nota} = 1,0 + \frac{\text{Puntaje obtenido}}{100} \times 6,0$$

Puntaje obtenido (%)	Nota
100	7,0
83	6,0
67	5,0
50	4,0 (mínimo para aprobar)
33	3,0
17	2,0
0	1,0

Cuadro 2: Tabla de conversión puntaje-nota